Práctica 2:

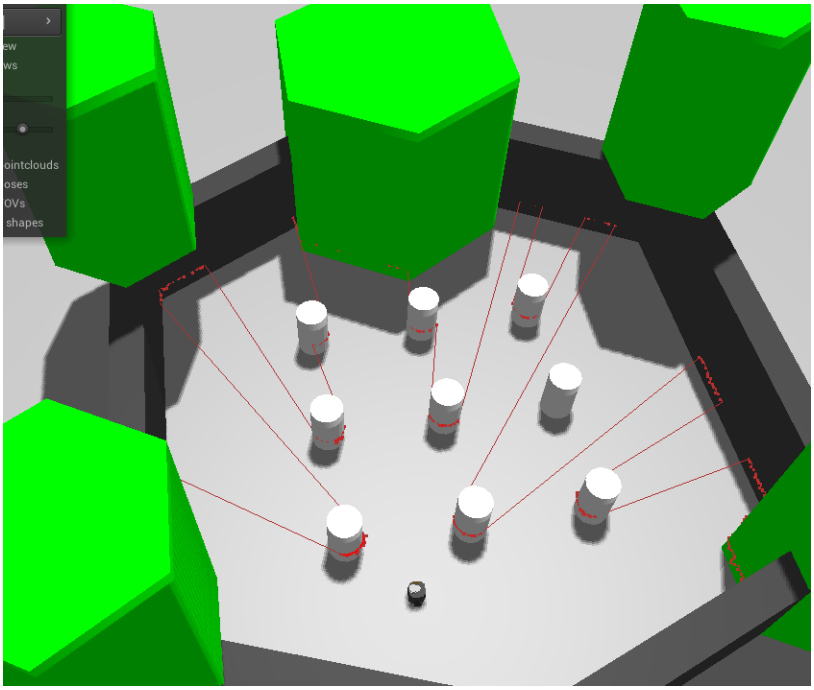
Mapeado, localización y navegación en Ros2

Para el desarrollo de esta práctica se hará uso del paquete que trae ros2: slam_toolbox. El cual cuenta con algoritmos de SLAM, que va generando el mapa mientras se va localizando.

Mapeado

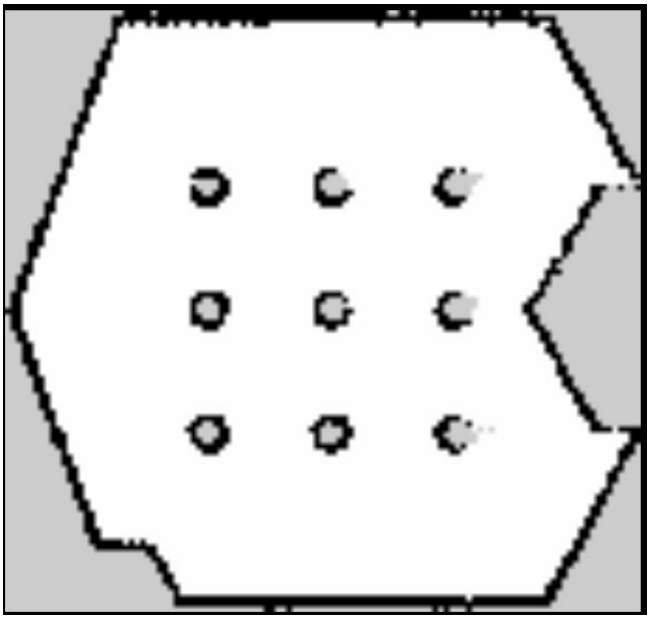
Para el primer caso, se usarán los mapas de tipo rejilla de ocupación.

Habiendo ejecutado el programa de SLAM de mapeado con "`ros2 launch slam_toolbox online_async_launch.py slam_params_file:=./slam.yaml` " y tras hacer un recorrido por el 'world' del turtrlebot que viene como demo con mvsim; se puede generar el mapa con este algoritmo de mapeado de rejilla.



Y al ejecutar "`ros2 run nav2_map_server map_saver_cli -f mapa`" se obtienen 2 archivos: mapa.pgm y mapa.yaml.

Nota: antes de ejecutarlo es necesario abrir un visor como rviz para comprobar que se ha hecho la detección completa del mapa.

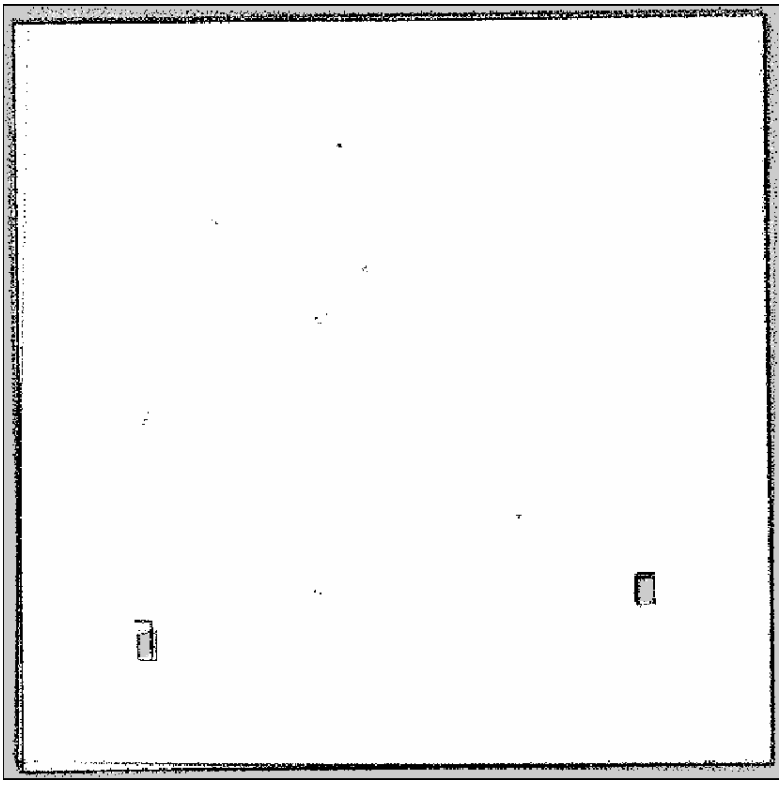


```yaml
image: mimapa.pgm
mode: trinary
resolution: 0.050
origin: [-2.885, -2.642, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.196
```

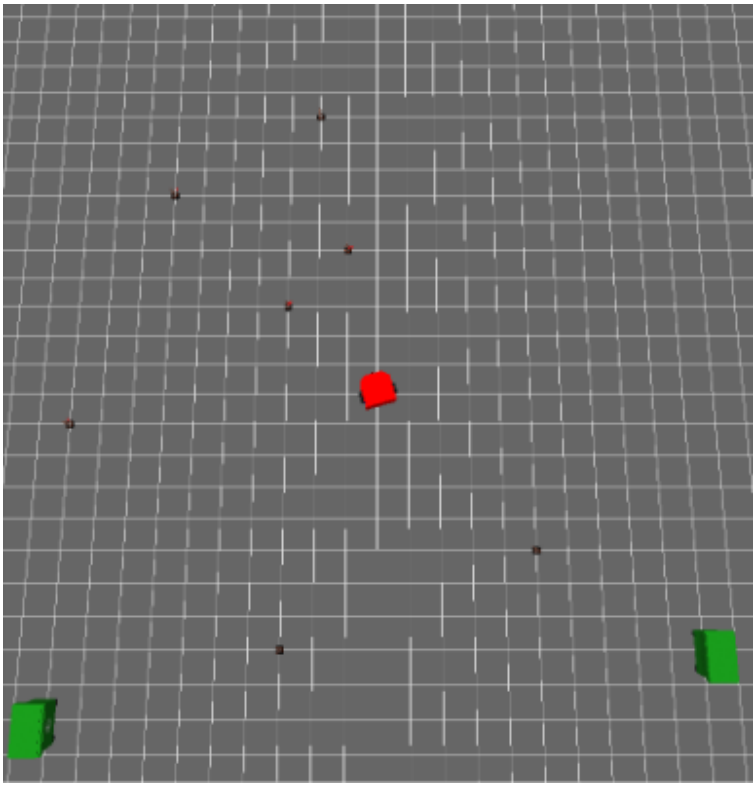
Parámetros del archivo mapa.yaml

- Image: hace referencia al nombre del archivo de imagen del mapa
- Mode: es la forma en la que se rellena cada casilla.
  - Trinary (3 valores): libre, ocupado o desconocido.
  - Scale: valores de gris según probabilidad.
  - Raw: valores decimales sin procesar.
- Resolution: tamaño real en metros que representa cada píxel (escala).
- Origin: posición (x,y,yaw) del origen del mapa.
- Negate: Si se invierten (1) o no (0) los colores del mapa.
- Occupied_thresh: umbral de ocupación para decidir si una celda se considera ocupada.
- Free_thresh: las celdas con un valor por debajo del definido se consideran libres.

Para el caso del mapeo del ‘world’ de la práctica 1 nos entrega la siguiente imágen .pgm:



El mapa real del que parte la detección es el de la siguiente figura:



Se aprecia que el mapa generado es bastante preciso pero que hay un cierto error en las medidas. Sobretodo se observa en el contenedor de abajo a la izquierda, en el cual los bordes no coinciden exactamente con la realidad y se encuentran desplazados.

Este fallo proviene del error de odometría que viene por defecto en el simulador y que es el tipo de medida que usa el algoritmo de SLAM para hacer la construcción del mapa.

El algoritmo de SLAM que incorpora slam_toolbox hace uso de la odometría y del lidar. Usa un método de grafos en el cual los nodos son las poses en las que el robot toma una medición y las aristas son las relaciones entre poses consecutivas o entre poses cercanas en un cierre de ciclo.

El algoritmo emplea el siguiente pipeline:

Se inicializa el modelo, se empieza a mover el robot el cual realiza escanéos con el láser y mediante las medidas de odometría se sabe cuál ha sido su desplazamiento. Cada uno de estos momentos se guarda como un nodo en el grafo de poses. Se añade una restricción con la pose anterior que viene dada por la odometría y un mecanismo de ICP.

Cuando se realiza lo que se conoce como cierre de ciclo, se produce una optimización global en el cual todas las poses anteriores se van actualizando como si fuera de manera recursiva; gracias a esto se consigue una mayor precisión. (visto en clase)

El mapa de ocupación se genera mediante un modelo bayesiano de celdas, las cuales tienen un valor que se va actualizando incrementalmente según las observaciones.

Localización

Para poder conocer la localización del robot se puede hacer uso del mapa generado anteriormente y así se evita el uso de la odometría.

Para ello se necesita generar un archivo de configuración ‘amcl.yaml’ .

```
amcl:
  ros__parameters:
    scan_topic: /laser1
```

Y se usará el algoritmo que implementa ROS basado en filtro de partículas AMLC. Este algoritmo muestra la posición y orientación del robot estimada a partir de un mapa conocido y de las lecturas del láser en el robot. Además, necesita una posición y orientación de partida estimada de forma manual, mediante 'Estimate pose 2D'.

[Video localización]

Parámetros de amcl

```
aitor@aitorpc:~/Escritorio/robots_moviles/p2$ ros2 param dump /amcl
/amcl:
  ros__parameters:
    /bond_disable_heartbeat_timeout: true
    alpha1: 0.2
    alpha2: 0.2
    alpha3: 0.2
    alpha4: 0.2
    alpha5: 0.2
    always_reset_initial_pose: false
    autostart_node: false
    base_frame_id: base_footprint
    beam_skip_distance: 0.5
    beam_skip_error_threshold: 0.9
    beam_skip_threshold: 0.3
    bond_heartbeat_period: 0.1
    do_beamskip: false
    first_map_only: false
    freespace_downsampling: false
    global_frame_id: map
    initial_pose:
      x: 0.0
      y: 0.0
      yaw: 0.0
      z: 0.0
    lambda_short: 0.1
    laser_likelihood_max_dist: 2.0
    laser_max_range: 100.0
    laser_min_range: -1.0
    laser_model_type: likelihood_field
    map_topic: map
    max_beams: 60
    max_particles: 2000
    min_particles: 500
    odom_frame_id: odom
    pf_err: 0.05
    pf_z: 0.99
```

```
    qos_overrides:
      /tf:
        publisher:
          depth: 100
          durability: volatile
          history: keep_last
          reliability: reliable
        subscription:
          depth: 100
          durability: volatile
          history: keep_last
          reliability: reliable
      /tf_static:
        subscription:
          depth: 100
          history: keep_last
          reliability: reliable
    recovery_alpha_fast: 0.0
    recovery_alpha_slow: 0.0
    resample_interval: 1
    robot_model_type: nav2_amcl::DifferentialMotionModel
    save_pose_rate: 0.5
    scan_topic: /laser1
    set_initial_pose: false
    sigma_hit: 0.2
    start_type_description_service: true
    tf_broadcast: true
    transform_tolerance: 1.0
    update_min_a: 0.2
    update_min_d: 0.25
    use_sim_time: false
    z_hit: 0.5
    z_max: 0.05
    z_rand: 0.5
    z_short: 0.05
```

En relación con el modelo de movimiento se encuentran los parámetros ***alpha***, que son las varianzas de rotación y translación proveniente de los errores de movimientos de rotación y translación. En este aspécto está relacionado directamente con lo visto en clase (tema 3.1 diapositivas 5-6).

En cuanto almodelo del sensor, los parámetros que modelan la observación y que hacen que AMCL evalúe las partículas ante las mediciones son los siguientes:

***z_hit, z_short, z_max, z_rand**:* referentes al **peso** de la componente gaussiana principal, cortas (obstáculo inesperado), fallo de sensor (max range) y aleatorio uniforme (medición errónea aleatoria).

También se implican las variables: ***sigma_hit, lambda_short, laser_model_type*** y ***laser_likelihood_max_dist*.**

En cuanto a los valores que modifican el filtro y las partículas aparecen los siguientes:

min_particles – max_particles: límites superior e inferior en el número de partículas

pf_err – pf_z: error máximo permitido en la estimación-- confianza mínima deseada

resample_interval: iteraciones entre remuestreo

recovery_alpha_slow: coeficientes de actualización adaptativa para la recuperación de localiazación.

Navegación

En cuanto a la navegación, se puede incorporar un sistema de planificación y seguimiento de trayectorias. A partir del mapa generado anteriormente y haciendo uso del AMCL; cuando damos una posición estimada del origen y un destino, el algoritmo genera una trayectoria para llevar al robot al destino sin necesidad de operarlo manualmente.

[Video navegación]

En el archivo “nav2.yaml” utiliza varios componentes de nivel superior:

```yaml
global_costmap:
  global_costmap:
    ros__parameters:
      global_frame: map
      robot_base_frame: base_link
      resolution: 0.05
      robot_radius: 0.20
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]
      static_layer:
        plugin: "nav2_costmap_2d::StaticLayer"
        map_topic: /map
        subscribe_to_updates: true
      obstacle_layer:
        plugin: "nav2_costmap_2d::ObstacleLayer"
        observation_sources: lidar
        lidar:
          topic: /laser1   # <--- TU topic real del laser
          data_type: LaserScan
          clearing: true
          marking: true
      inflation_layer:
        plugin: "nav2_costmap_2d::InflationLayer"
        inflation_radius: 0.6
```

El global_costmap es una representación del mapa en forma de cuadrícula el cual es utilizado por el *global_planner* para planificar rutas completas.

Se va a ir explicando el código de arriba a abajo.

Map indica que se usa le marco de coordenadas del mapa global y para el robot se usa base_link que es el marco de referencia del robot. Se presupone una transdormación válida entre ambos sistemas.

En cuanto a *resolution* representa el tamaño en metros con respecto al mundo real, siendo 5 cm x 5 cm y el *robot_radius* 20cm de radio (usado para evitar obstáculos).

Para un mapa más detallado se puede disminuir la resolución.

Los *plugins* indican las capas que se utilizan para representar el costmap, las cuales se combinan para crear una sola cuadrícula con costes ajustados.

Ahora se definen cada una de estas 3 capas:

Static_layer: obtiene el mapa estático del entorno y se actualiza si hay cambios.

Obstacle_layer: represeta los obstáculos de forma dinámica mediante el uso de los sensores y va actualizando según las lecuras.

Inflation_layer: expande los obstáculos para crear zonas de seguridad a su alrededor sobre las cuales el robot no debería entrar y por tanto la trayectoria las exluirán. Este parámetro se puede variar para ser más permisivo en cuanto a la cercanía a los objetos o más seguro para que se aleje más. Si se reduce el valor la seguridad disminuye pero si se aumenta demasiado pueden aparecer zonas las cuales sean innaccesibles cuando realmente el robot si podría llegar. Para entornos estrechos convendría disminuir el valor para que no se generen problemas de trayectoria ni puntos innaccesibles.

```yaml
controller_server:
  ros__parameters:
    use_sim_time: false
    controller_plugins: ["FollowPath"]
    FollowPath:
      plugin: "nav2_regulated_pure_pursuit_controller::RegulatedPurePursuitController"
      desired_linear_vel: 0.4
      lookahead_dist: 0.6
```

El *controller_server* ejecta el control local, el cual convierte el camino global en comandos de movimiento (velocidad lineal y angular).

En este caso se usa el tiempo real y no el de simulación (*use_sim_time= false*).

“FollowPath” hace referencia al tipo de controlador que se va a utilizar:

El plugin que utiliza es el más utilizado el cual toma el camino global, calcula un punto delante del robot y genera comandos de velocidad para dirigirlo suavemente.

Si quisieramos aumentar la velocidad de movimiento podríamos aumentar el parámetro *desired_linear_vel*. Para un control más preciso de las posiciones se puede disminuir *lookahead_dist*.